Universidad Metropolitana

Departamento de Energía y Automatización

Robótica Industrial

PRÁCTICA Nº 6

Dobot Magician

Asignatura: Robótica Industrial (FPTEN27)

Profesor: José Montilva

Laboratorio de Robótica Industrial

Caracas, noviembre de 2026

Guía de Práctica de Laboratorio

Práctica #6 — Dobot Magician: Pick & Place y trayectorias

Objetivo General

Programar el brazo robótico Dobot Magician con tres métodos de complejidad creciente — enseñanza manual (Teaching & Playback), bloques (DobotBlock Lab) y Python (`pydobot`)— para resolver tareas de Pick & Place y de trazado de trayectorias, relacionando cada método con la cinemática y la generación de trayectorias vistas en clase.

Antes de empezar — El Dobot Magician

El Dobot Magician es un brazo robótico serial de sobremesa con 4 grados de libertad (GDL). Pertenece a la familia de configuraciones antropomórficas: sus articulaciones imitan la estructura de hombro, codo y muñeca de un brazo humano, aunque simplificado para operar sobre un plano.

Conexión con Tema 1 — Definición y morfología de robots

Recuerda la clasificación de robots vista en clase. El Dobot es un manipulador serial de cadena cinemática abierta: cada articulación depende de la posición de la anterior. Esto lo diferencia de robots paralelos o cartesianos.

El robot opera en un sistema de coordenadas cartesianas (X, Y, Z) expresadas en milímetros, más un ángulo de rotación de herramienta (R) en grados:

- **X** → distancia frontal desde la base (0 mm = base del robot)
- **Y** → desplazamiento lateral (negativo = derecha, positivo = izquierda)
- **Z** → altura (valores negativos = por debajo del nivel de la base)
- **R** → rotación de la herramienta sobre el eje vertical (0° = frente)

Conexión con Tema 8 — Cinemática de robots

Cada vez que ejecutas `pose()` en Python, el robot está resolviendo su cinemática directa internamente: a partir de los ángulos de sus articulaciones (j1, j2, j3, j4), calcula la posición cartesiana (X, Y, Z) del extremo de la herramienta.

El Dobot admite tres herramientas intercambiables según la tarea:

- **Ventosa de succión (Suction Cup):** para Pick & Place de objetos planos o lisos.
- **Portaminas / lápiz (Pen attachment):** para trazado y escritura sobre papel.
- **Gripper (pinza):** para manipulación de objetos de geometría irregular.

El botón físico de desbloqueo (**Unlock**) se encuentra en la parte delantera del brazo. Al mantenerlo presionado, los motores se desenganchan y el brazo puede moverse libremente con la mano. Se usa en los Ejercicios #1 y #3.

Normas de seguridad

⚠ Lee estas normas antes de encender el robot

- Mantén manos, cabello y ropa alejados de las partes móviles en todo momento.
- Nunca fuerces el brazo contra sus topes mecánicos: los servos pueden dañarse.
- Desconecta la alimentación antes de cambiar la herramienta (ventosa ↔ lápiz).
- Al usar el botón Unlock para mover el brazo manualmente, hazlo con suavidad y movimientos lentos.
- Mantén la mano cerca del cable de alimentación al ejecutar el primer movimiento: si el robot se mueve de forma inesperada, desconecta inmediatamente.
- Al instalar el lápiz, asegúrate de que el resorte interno tenga recorrido libre; si está demasiado ajustado, los servomotores trabajarán contra la mesa.
- No manipules la ventosa mientras el robot esté en movimiento: podría soltar o capturar un objeto inesperadamente.

Preparación del entorno

Puerto COM. El Dobot se conecta al computador por USB y Windows le asigna un puerto COM. Para encontrarlo:

1. Abre el **Administrador de dispositivos** de Windows.
2. Expande la sección **Puertos (COM y LPT)**.
3. Busca el dispositivo **USB-SERIAL** o similar y anota el número (ej.: COM7).
4. Reemplaza `COMX` por ese número en todos los scripts de Python de esta práctica.

⚠ Puerto incorrecto = error de conexión

- El número de puerto puede cambiar si conectas el cable a otro puerto USB.

Librerías de Python. Abre una terminal (CMD o PowerShell) y ejecuta, una sola vez por computador:

```
pip install pydobot pyserial
pip install Hershey-Fonts
```

¿Qué instala cada librería?

- `pydobot`: comunicación con el robot (protocolo serial propietario de Dobot).
- `pyserial`: manejo del puerto COM a bajo nivel.
- `Hershey-Fonts`: fuentes vectoriales de los años 60, diseñadas originalmente para plotters y máquinas CNC.

Ejercicio #1 Teaching & Playback

En esta primera parte resolverás el problema de Pick & Place de la forma más intuitiva: moviendo el robot con la mano para "enseñarle" las posiciones. El robot grabará cada punto y lo reproducirá exactamente.

Analogía

Es como grabar un video de ti mismo haciendo una tarea y luego presionar "play". El robot no entiende qué hace ni por qué: simplemente repite los movimientos grabados. Si cambias los cubos de lugar, el robot seguirá yendo a las posiciones antiguas.

Conexión con Tema 9 — Generación de trayectorias

Teaching & Playback genera exclusivamente trayectorias PTP (punto a punto): el robot memoriza una secuencia de coordenadas y las recorre en orden fijo.

Paso 1 — Acceder al módulo de enseñanza

1. Abre **DobotLab** desde el navegador (o la aplicación de escritorio).
2. En la pantalla principal, selecciona el módulo **Teaching & Playback**.
3. Conecta el Dobot y verifica que la conexión sea exitosa.



Módulo Teaching & Playback en DobotLab

Paso 2 — Desbloquear el brazo

Mantén presionado el botón **Unlock**. Mientras lo sostienes, puedes desplazar las articulaciones libremente. Al soltarlo, los motores vuelven a enganchar y la posición queda fija. Mueve el brazo con suavidad y sin forzarlo contra sus topes.

Paso 3 — Grabar la secuencia

Grabarás las posiciones clave para recoger un cubo y colocarlo en otra posición. Repite para al menos 3 cubos:

1. Con el brazo desbloqueado, lleva la ventosa encima del primer cubo y baja hasta tocarlo. En la interfaz, activa la ventosa (**Suction Cup ON**).
2. Sube el brazo con el cubo a una altura segura, desplázate al destino (punto B) y baja. Desactiva la ventosa (**Suction Cup OFF**).
3. Sube el brazo para alejarte del cubo colocado.
4. Repite el procedimiento para los cubos 2 y 3.

Paso 4 — Reproducir

Presiona **Play**. El robot repetirá exactamente la secuencia grabada. Observa si logra apilar los cubos correctamente.

⚠ Observa y anota

- Cuenta cuántos puntos tuviste que grabar en total para mover 3 cubos. En el Ejercicio #2, un solo ciclo con fórmulas moverá 6 cubos sin grabar ningún punto.
- Desplaza uno de los cubos 2 cm y vuelve a reproducir. ¿Qué ocurre?

Ejercicio #2 DobotBlock Lab: Pick & Place algorítmico

Ahora resolverás el mismo problema de forma algorítmica. En lugar de grabar posiciones, le darás al robot ecuaciones que calculan automáticamente dónde está cada cubo en una cuadrícula de 2 filas × 3 columnas.

Analogía

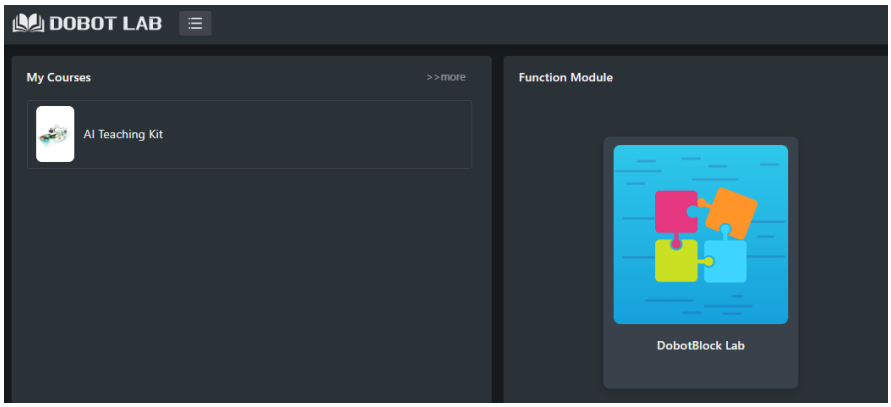
Imagina un cartero con 6 buzones en 2 filas × 3 columnas. En lugar de memorizar cada dirección, usa "fila = bloque ÷ 3" y "columna = residuo de bloque ÷ 3". Con un solo contador (0 a 5) sabe exactamente a cuál ir. El robot hace lo mismo.

Conexión con Tema 8 — Cinemática de robots

Un único número entero (`block_pos`) codifica una posición en un espacio bidimensional discreto: es una representación de posición en un espacio cartesiano discretizado.

Paso 0 — Acceder a DobotBlock Lab

1. Vuelve a la pantalla principal de DobotLab.
2. Selecciona **DobotBlock Lab**.
3. Conecta el robot.



Pantalla principal de DobotLab — módulo DobotBlock Lab

Paso 1 — Crear variables

En la categoría **Variables** crea estas 5 variables:

- `block_side` → tamaño del cubo en mm (ej.: 26)
- `block_pos` → contador de posición (de 0 a 5)
- `x`, `y`, `z` → coordenadas actuales del robot



Variables creadas

Paso 2 — Controles de teclado

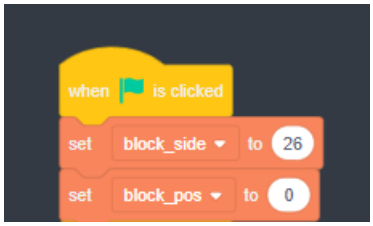
1. En **Events** arrastra dos bloques **When [key] pressed**.
2. En el primero asigna la tecla **[space]** y coloca debajo **Suction Cup [Off]**. Es el botón de emergencia: al presionar espacio, la ventosa suelta lo que tenga agarrado.
3. En el segundo asigna **[flecha abajo ↓]** y coloca **set [x] to [Get Pose X]**, **set [y] to [Get Pose Y]** y **set [z] to [Get Pose Z]**. Así registras las coordenadas actuales del robot en las variables.



Eventos de teclado

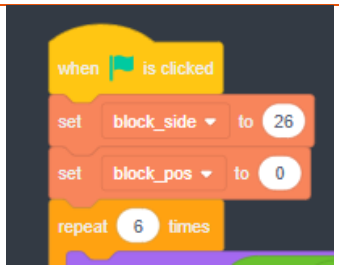
Paso 3 — Bloque principal y valores iniciales

En **Events** arrastra **When [bandera verde] clicked** y, debajo, **set [block_side] to 26** y **set [block_pos] to 0**.



Paso 4 — El ciclo de repetición

Arrastra **repeat [6] times** debajo de las variables. Todo lo que sigue va dentro de este bloque.



Paso 5 — Ecuaciones de la posición de recogida

Dentro del repeat, calcula las coordenadas de cada cubo con estas ecuaciones y encaja los bloques matemáticos en los campos de **Jump to**:

```
Y = 179 + (block_side × (block_pos % 3))
X = 167 − (block_side × (block_pos ÷ 3))
Z = −38   (fijo: todos los cubos están sobre la mesa)
R = 0
```

¿Cómo funciona?

El operador módulo (%) da la columna: 0%3=0, 1%3=1, 2%3=2, 3%3=0… La división entera da la fila: 0÷3=0, 3÷3=1. Un solo número (block_pos de 0 a 5) codifica fila y columna sin grabar ninguna posición.

Paso 6 — Movimiento de recogida

Después de las ecuaciones arrastra **Jump to X Y Z R** con los valores calculados y, debajo, **Suction Cup ON**.

Paso 7 — Movimiento de apilado

Todos los cubos van a la misma posición XY, pero apilados: cada cubo nuevo queda encima del anterior, por lo que Z aumenta en cada iteración:

```
Jump to  X: 241   Y: −66   Z: −38 + (block_side × block_pos)   R: 0
Suction Cup → OFF
```

¿Por qué cambia Z?

Bloque 0 → Z = −38 (nivel de la mesa). Bloque 1 → Z = −38 + 26 = −12. Bloque 2 → Z = −38 + 52 = 14. Es como subir una escalera donde cada peldaño tiene la altura exacta de un cubo.

⚠ Coordenadas de ejemplo

• Los valores X = 241 e Y = −66 son de referencia. Verifica las coordenadas reales de tu posición de apilado con la tecla ↓ mientras el brazo está sobre el destino.

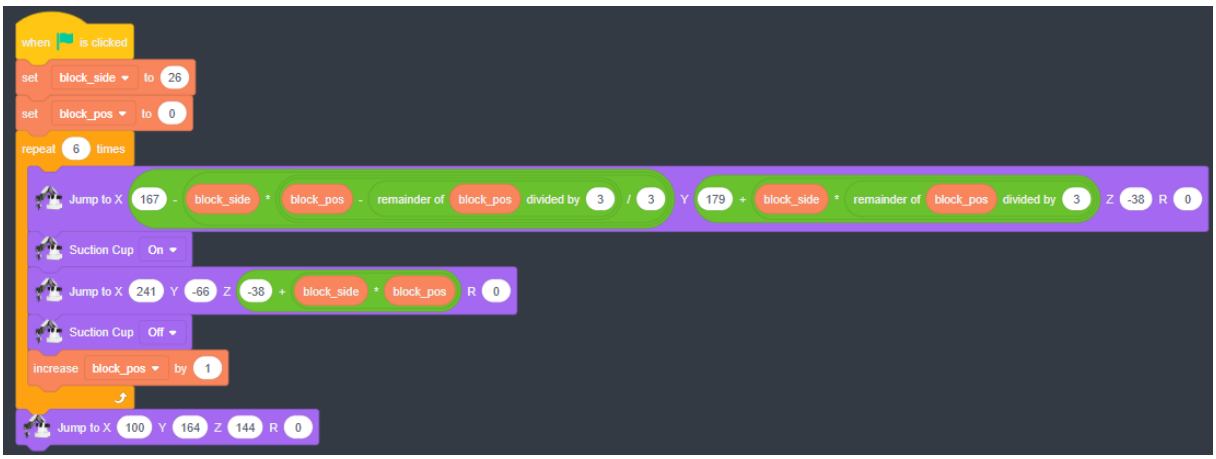


Valores de las variables en el panel del robot

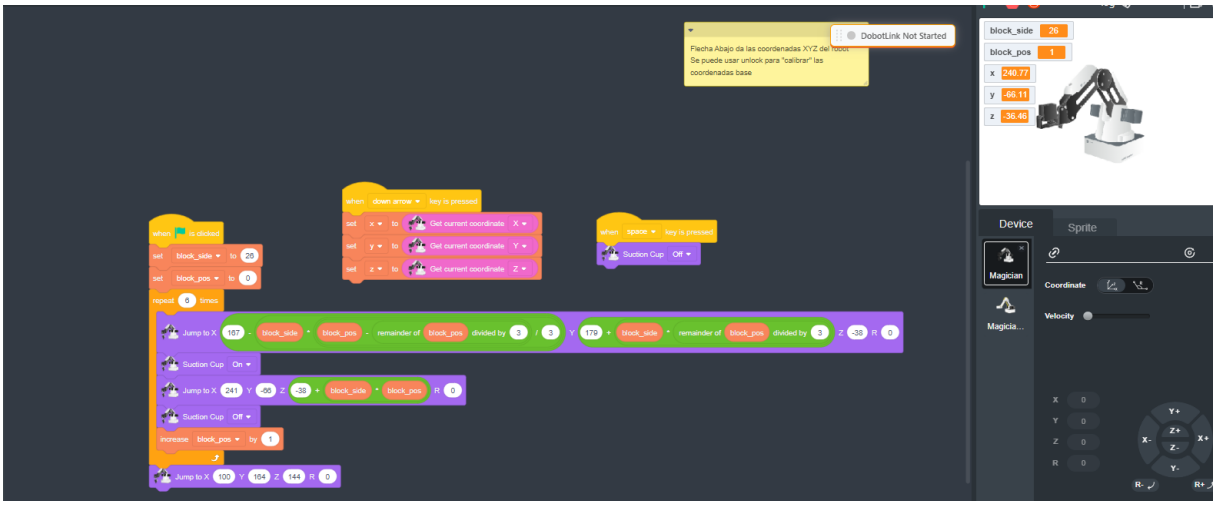
Paso 8 — Actualizar el contador y finalizar

1. Último bloque dentro del repeat: **increase [block_pos] by 1**.
2. Fuera del repeat: **Jump to X [100] Y [164] Z [144] R [0]** (posición segura de reposo).

Programa completo:



Vista del workspace:



Ejercicio #3 Python: calibración y trazado del laberinto

Ahora el robot se programa desde Python con `pydobot`. Primero registrarás las coordenadas de los puntos clave de un laberinto impreso y luego programarás al robot para trazar su solución con el lápiz.

Analogía

Antes de que un piloto vuele una ruta nueva, registra los waypoints en el GPS. La calibración hace lo mismo: convierte puntos físicos del papel en coordenadas numéricas que el robot puede seguir.

Conexión con Temas 8 y 10 — Cinemática y lenguajes de robótica

`pose()` resuelve la cinemática directa del robot en tiempo real: a partir de los ángulos de sus 4 articulaciones calcula la posición (X, Y, Z) del extremo, es decir, el espacio cartesiano o de la herramienta.

Preparación física

1. Fija la hoja con el laberinto sobre la mesa, frente al robot, alineada y plana.
2. Retira la ventosa e instala el lápiz. Verifica que el resorte tenga recorrido libre.
3. Anota tu puerto COM del Administrador de dispositivos.

Paso 1 — Calibracion.py

Crea un archivo llamado `Calibracion.py` con el siguiente contenido y reemplaza `COMX` por tu puerto real. Mientras se ejecuta, lleva el lápiz a cada punto con el botón Unlock y pulsa ENTER en la terminal.

```
# ============================================================
# Calibracion.py
# Práctica 6 — Dobot Magician | Robotica Industrial (FPTEN27)
# UNIMET — Departamento de Energia y Automatizacion
#
# Proposito: Leer y registrar las coordenadas cartesianas
#            (X, Y, Z) de puntos clave del laberinto.
#
# ANTES DE EJECUTAR:
#   1. Instala las librerias: pip install pydobot pyserial
#   2. Conecta el Dobot por USB y enciendelo.
#   3. Verifica el puerto COM en el Administrador de Dispositivos
#      (Windows > Puertos COM y LPT > USB-SERIAL).
#   4. Reemplaza 'COMX' por tu puerto real (ej: 'COM7').
#
# COMO MOVER EL BRAZO MANUALMENTE:
#   El Dobot tiene un boton fisico de desbloqueo (Unlock button)
#   en la parte delantera del brazo. Mantenlo presionado mientras
#   mueves el brazo con la mano. Sueltalo para fijar la posicion.
#   El script NO mueve ni desbloquea el robot — solo lee la posicion.
# ============================================================

from pydobot import Dobot

PUERTO = 'COMX'   # <-- Cambia esto segun tu configuracion
```

```
# Lista de puntos a calibrar en orden
PUNTOS = [
    "Entrada del laberinto",
    "Giro 1",
    "Giro 2",
    "Giro 3",
    "Salida del laberinto",
    "Inicio de escritura del nombre",
]

print("=" * 55)
print("  CALIBRADOR DE COORDENADAS — Dobot Magician")
print("=" * 55)
print()

# Conectar al Dobot
try:
    robot = Dobot(port=PUERTO, verbose=False)
    print(f"Conexion exitosa en {PUERTO}")
except Exception as e:
    print(f"Error al conectar: {e}")
    print("Verifica el puerto COM y que el Dobot este encendido.")
    exit(1)

print()
print("INSTRUCCIONES:")
print("  - Mantén presionado el boton Unlock del brazo.")
print("  - Mueve el lapiz hasta el punto indicado.")
print("  - Suelta el boton y presiona ENTER en la terminal.")
print()

coordenadas = {}

for punto in PUNTOS:
    input(f">> Ubica el lapiz en: [{punto}]  y presiona ENTER...")

    # pose() devuelve: (x, y, z, r, j1, j2, j3, j4)
    # x, y, z: posicion cartesiana en mm
    # r: rotacion de la herramienta en grados
    # j1-j4: angulos articulares (calculados internamente por el robot)
    pose = robot.pose()
    x, y, z = pose[0], pose[1], pose[2]

    coordenadas[punto] = (x, y, z)
    print(f"   Registrado -> X={x:.2f} mm  |  Y={y:.2f} mm  |  Z={z:.2f} mm")
    print()

# Mostrar resumen
print("=" * 55)
print("  RESUMEN DE COORDENADAS REGISTRADAS")
print("=" * 55)
for nombre, (x, y, z) in coordenadas.items():
    print(f"  {nombre}:")
    print(f"    X={x:.2f}, Y={y:.2f}, Z={z:.2f}")
print()
print("Copia estos valores en Draw.py y Write.py.")
```

```
robot.close()
print("Conexion cerrada.")
```

Paso 2 — Draw.py

Con las coordenadas calibradas, el robot recorrerá la solución del laberinto. Levanta el lápiz (`Z_ALTO`) para desplazarse y lo baja (`Z_ABAJO`) solo cuando debe dejar marca. Crea `Draw.py` y reemplaza los espacios en blanco con tus coordenadas:

Conexión con Temas 9 y 12 — Trayectorias y path planning

Esta secuencia de puntos es una planificación de ruta offline: calculaste el camino antes de ejecutarlo. Contrasta con el Teaching & Playback del Ejercicio #1, donde el robot solo repite posiciones grabadas. En path planning (Tema 12), algoritmos como A* o RRT calculan esta ruta automáticamente.

```
# ============================================================
# Draw.py
# Práctica 6 — Dobot Magician | Robotica Industrial (FPTEN27)
# UNIMET — Departamento de Energia y Automatizacion
#
# Proposito: Trazar la ruta correcta de un laberinto impreso
#            moviendo el lapiz entre los puntos calibrados.
#
# ANTES DE EJECUTAR:
#   1. Ejecuta primero Calibracion.py y anota las coordenadas.
#   2. Reemplaza los valores de HOME, Z_ALTO, Z_ABAJO y
#      los puntos del camino con tus coordenadas reales.
#   3. Reemplaza 'COMX' por tu puerto real (ej: 'COM7').
#
# LOGICA DE LAPIZ:
#   Z_ALTO  = altura de desplazamiento (lapiz en el aire)
#   Z_ABAJO = altura de trazado (lapiz tocando el papel)
#   El robot siempre viaja entre puntos con el lapiz arriba,
#   y solo baja cuando debe dejar marca sobre el papel.
# ============================================================

from pydobot import Dobot

PUERTO = 'COMX'   # <-- Cambia esto segun tu configuracion

# --- Posicion segura de inicio y fin ---
HOME_X = 200.0    # <-- Reemplaza con tu HOME real
HOME_Y =   0.0
HOME_Z = 100.0

# --- Alturas de trabajo (obtenidas en Calibracion.py) ---
Z_ALTO  = ______   # <-- Altura con lapiz en el aire (ej: -20.0)
Z_ABAJO = ______   # <-- Altura con lapiz tocando el papel (ej: -55.0)

# --- Camino del laberinto (X, Y) en orden: entrada -> giros -> salida ---
# Reemplaza cada par con las coordenadas obtenidas en Calibracion.py
camino = [
```

```
    (_______, _______),   # Entrada del laberinto
    (_______, _______),   # Giro 1
    (_______, _______),   # Giro 2
    (_______, _______),   # Giro 3
    (_______, _______),   # Salida del laberinto
]

# ============================================================
print("Conectando al Dobot...")
try:
    robot = Dobot(port=PUERTO, verbose=False)
    print("Conexion exitosa.")
except Exception as e:
    print(f"Error al conectar: {e}")
    exit(1)

robot.speed(velocity=80, acceleration=80)

# 1. Ir a posicion segura HOME
print("Yendo a HOME...")
robot.move_to(HOME_X, HOME_Y, HOME_Z, 0, wait=True)

# 2. Desplazarse sobre la entrada del laberinto con lapiz ARRIBA
x_entrada, y_entrada = camino[0]
print("Desplazandose sobre la entrada...")
robot.move_to(x_entrada, y_entrada, Z_ALTO, 0, wait=True)

# 3. Bajar el lapiz en la entrada (inicio del trazado)
print("Bajando lapiz en la entrada...")
robot.move_to(x_entrada, y_entrada, Z_ABAJO, 0, wait=True)

# 4. Trazar el camino: moverse por cada punto con lapiz ABAJO
print("Trazando el laberinto...")
for i, (x, y) in enumerate(camino[1:], start=1):
    print(f"  Punto {i}: X={x:.1f}, Y={y:.1f}")
    robot.move_to(x, y, Z_ABAJO, 0, wait=True)

# 5. Levantar el lapiz al llegar a la salida
x_salida, y_salida = camino[-1]
print("Levantando lapiz en la salida...")
robot.move_to(x_salida, y_salida, Z_ALTO, 0, wait=True)

# 6. Regresar a HOME
print("Regresando a HOME...")
robot.move_to(HOME_X, HOME_Y, HOME_Z, 0, wait=True)

robot.close()
print("Trazado completado. Conexion cerrada.")
```

⚠ ¿Qué observar?

- ¿Qué pasa con el trazo si `Z_ABAJO` es demasiado bajo o demasiado alto?
- Compara las coordenadas que registraste con las de otro grupo: ¿por qué no son iguales si el laberinto es el mismo?

Ejercicio #4 Python: escritura con HersheyFonts

El robot escribirá el nombre de un integrante del equipo usando la librería HersheyFonts, que contiene fuentes vectoriales diseñadas en los años 60 para plotters y máquinas CNC, y que sigue usándose en la industria.

Analogía

HersheyFonts es un "molde de galletas digital": cada letra es una receta de segmentos de línea con coordenadas. La función `lines_for_text()` descompone el texto en esos segmentos, igual que un plotter industrial, y el robot traza cada segmento subiendo y bajando el lápiz entre ellos.

Conexión con Temas 9 y 11 — Trayectorias y visión artificial

Cada letra es una secuencia de trayectorias PTP encadenadas (Tema 9). Además, en la clase de visión artificial viste `pipeline_dobot.py`, que convierte coordenadas en píxeles de una cámara a milímetros del robot: el mismo tipo de transformación de espacio que hace aquí HersheyFonts al convertir coordenadas de la fuente a milímetros reales.

Crea `Write.py`, reemplaza `TEXTO` por el nombre del integrante (en mayúsculas) y los valores de posición por los obtenidos en `Calibracion.py`:

```
# ============================================================
# Write.py
# Práctica 6 — Dobot Magician | Robotica Industrial (FPTEN27)
# UNIMET — Departamento de Energia y Automatizacion
#
# Proposito: Escribir el nombre de un integrante del equipo
#            usando la libreria HersheyFonts para convertir
#            texto en trayectorias de trazado fisico.
#
# ANTES DE EJECUTAR:
#   1. Instala: pip install Hershey-Fonts
#   2. Copia Z_ALTO y Z_ABAJO de tu Draw.py ya verificado.
#   3. Ajusta X_INICIO e Y_INICIO para que el texto quede
#      dentro del area de trabajo del robot (ver Calibracion.py).
#   4. Cambia TEXTO por el nombre real del integrante.
#   5. Reemplaza 'COMX' por tu puerto real (ej: 'COM7').
#
# COMO FUNCIONA HersheyFonts:
#   La libreria Hershey contiene fuentes vectoriales usadas
#   originalmente en plotters y maquinas CNC desde los años 60.
#   lines_for_text() descompone cada letra en segmentos de linea:
#     cada segmento = ((x0, y0), (x1, y1))
#   El robot traza cada segmento levantando el lapiz entre ellos.
#   Es el mismo principio que usa un plotter industrial.
#
# NOTA SOBRE EL EJE Y:
#   Las coordenadas Y de HersheyFonts usan convenio de pantalla
#   (Y positivo = hacia abajo). Para que las letras no queden
#   invertidas en el papel, se aplica: y_robot = Y_INICIO - y_fuente * ESCALA
# ============================================================
```

```
from pydobot import Dobot
from HersheyFonts import HersheyFonts

PUERTO  = 'COMX'     # <-- Cambia esto segun tu configuracion
TEXTO   = 'NOMBRE'   # <-- Cambia por el nombre del integrante (mayusculas)

# --- Posicion segura ---
HOME_X = 200.0       # <-- Reemplaza con tu HOME real
HOME_Y =   0.0
HOME_Z = 100.0

# --- Alturas de trabajo (mismas que en Draw.py) ---
Z_ALTO  = ______      # <-- Lapiz en el aire
Z_ABAJO = ______      # <-- Lapiz tocando el papel

# --- Posicion inicial del texto en el papel ---
X_INICIO =  ______    # <-- Coordenada X donde comienza la primera letra
Y_INICIO =  ______    # <-- Coordenada Y central del texto

# --- Escala del texto (0.8 = letras de ~17 mm de alto) ---
ESCALA = 0.8

# ============================================================

def escribir_texto(robot, texto, x_inicio, y_inicio, escala, z_alto, z_abajo):
    """
    Traza el texto dado sobre el papel usando el lapiz del Dobot.

    Parametros:
        robot     : instancia conectada de Dobot
        texto     : cadena de texto a escribir (str)
        x_inicio  : coordenada X del extremo izquierdo del texto (mm)
        y_inicio  : coordenada Y central del texto (mm)
        escala    : factor de escala de la fuente (float)
        z_alto    : altura con lapiz en el aire (mm)
        z_abajo   : altura con lapiz tocando el papel (mm)

    Como funciona internamente:
        HersheyFonts descompone cada letra en segmentos de linea.
        Cada segmento es una tupla: ((x0, y0), (x1, y1)).
        Para trazar un segmento:
          1. Nos desplazamos al punto inicial (x0, y0) con el lapiz arriba.
          2. Bajamos el lapiz al papel.
          3. Nos desplazamos al punto final (x1, y1) trazando.
          4. Levantamos el lapiz.
        El eje Y se invierte porque HersheyFonts usa convenio de pantalla
        (Y hacia abajo) mientras que el robot usa Y hacia el frente.
    """
    fuente = HersheyFonts()
    fuente.load_default_font()

    # lines_for_text devuelve un iterador de segmentos: ((x0,y0),(x1,y1))
    segmentos = list(fuente.lines_for_text(texto))
    total = len(segmentos)
    print(f"Texto '{texto}': {total} segmentos a trazar.")
```

```
    for i, segmento in enumerate(segmentos):
        p_inicio, p_fin = segmento   # cada punto es (x, y) en unidades Hershey

        # Convertir coordenadas Hershey → espacio fisico del Dobot
        # Inversion de Y: y_robot = y_inicio - y_hershey * escala
        x0 = x_inicio + p_inicio[0] * escala
        y0 = y_inicio - p_inicio[1] * escala
        x1 = x_inicio + p_fin[0]   * escala
        y1 = y_inicio - p_fin[1]   * escala

        # Paso 1: desplazarse al inicio del segmento con lapiz ARRIBA
        robot.move_to(x0, y0, z_alto, 0, wait=True)

        # Paso 2: bajar el lapiz
        robot.move_to(x0, y0, z_abajo, 0, wait=True)

        # Paso 3: trazar hacia el final del segmento
        robot.move_to(x1, y1, z_abajo, 0, wait=True)

        # Paso 4: levantar el lapiz
        robot.move_to(x1, y1, z_alto, 0, wait=True)

        if (i + 1) % 10 == 0:
            print(f"  Progreso: {i + 1}/{total} segmentos")

    print(f"Escritura de '{texto}' completada.")


# ============================================================
print("Conectando al Dobot...")
try:
    robot = Dobot(port=PUERTO, verbose=False)
    print("Conexion exitosa.")
except Exception as e:
    print(f"Error al conectar: {e}")
    exit(1)

robot.speed(velocity=60, acceleration=60)

# 1. Ir a posicion segura HOME
print("Yendo a HOME...")
robot.move_to(HOME_X, HOME_Y, HOME_Z, 0, wait=True)

# 2. Escribir el texto
escribir_texto(
    robot    = robot,
    texto    = TEXTO,
    x_inicio = X_INICIO,
    y_inicio = Y_INICIO,
    escala   = ESCALA,
    z_alto   = Z_ALTO,
    z_abajo  = Z_ABAJO
)

# 3. Regresar a HOME
```

```
print("Regresando a HOME...")
robot.move_to(HOME_X, HOME_Y, HOME_Z, 0, wait=True)

robot.close()
print("Escritura finalizada. Conexion cerrada.")
```

⚠ Ajuste de escala

- `ESCALA = 0.8` produce letras de aproximadamente 17 mm de alto. Si el nombre es largo y se sale del área de trabajo, reduce `ESCALA` o ajusta `X_INICIO`.
- Escribe el nombre de **cada integrante** del grupo, cambiando solo `TEXTO`.

Al finalizar

1. Lleva el brazo a la posición de reposo y desconecta el robot del puerto USB.
2. Apaga el Dobot, retira el lápiz y deja la ventosa instalada para el siguiente grupo.
3. Recoge las hojas, los cubos y deja el puesto ordenado.